

Министерство образования Республики Беларусь  
Учреждение образования  
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет: Инженерно-экономический

Кафедра: Экономической информатики

**ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1**  
**на тему:**

Выполнил

Филяковский С. В.  
гр. 377901

Проверила

Семенюк А. В.

Минск 2026

## Вариант 12

Комплексная оптимизация запросов к таблице заказов интернет-магазина.

Требуется:

- создать не менее **4 таблиц**:
  - заказы;
  - клиенты;
  - товары;
  - позиции заказа;
- реализовать запрос аналитического отчёта;
- выявить проблемы производительности;
- применить:
  - индексы;
  - партиционирование;
  - переписывание SQL;
  - сокращение количества JOIN;
- сравнить результаты EXPLAIN ANALYZE;
- обосновать, какой набор оптимизаций дал максимальный эффект.

## Пример

```
SELECT c.name, SUM(oi.quantity * oi.price) AS total_sum
FROM customers c
JOIN orders o ON o.customer_id = c.id
JOIN order_items oi ON oi.order_id = o.id
GROUP BY c.name;
```

## docker-compose

services:

postgres:

image: postgres:16  
container\_name: postgres  
restart: no

ports:  
- "5432:5432"

volumes:  
- postgres\_data:/var/lib/postgresql/data/  
- ./init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro

environment:  
POSTGRES\_USER: \${POSTGRES\_USER}  
POSTGRES\_PASSWORD: \${POSTGRES\_PASSWORD}  
POSTGRES\_DB: ApplicationDatabase

healthcheck:  
test: ["CMD-SHELL", "pg\_isready -U \${POSTGRES\_USER}"]

interval: 10s  
timeout: 5s  
retries: 5

volumes:  
postgres\_data:

init

```
-- =====
```

```
-- Создание структуры БД интернет-магазина  
-- =====
```

```
-- 1. Клиенты  
CREATE TABLE IF NOT EXISTS clients (  
id SERIAL PRIMARY KEY,  
name VARCHAR(100) NOT NULL,  
email VARCHAR(255) UNIQUE NOT NULL,  
phone VARCHAR(20),  
created_at TIMESTAMP DEFAULT now()  
);
```

```
-- 2. Товары  
CREATE TABLE IF NOT EXISTS products (  
id SERIAL PRIMARY KEY,  
name VARCHAR(200) NOT NULL,  
price NUMERIC(10,2) NOT NULL,  
category VARCHAR(100)  
);
```

```
-- 3. Заказы  
CREATE TABLE IF NOT EXISTS orders (  
id SERIAL PRIMARY KEY,  
client_id INT REFERENCES clients(id),  
order_date TIMESTAMP DEFAULT now(),  
status VARCHAR(20) DEFAULT 'new',  
total_amount NUMERIC(12,2)  
);
```

```
-- 4. Позиции заказа  
CREATE TABLE IF NOT EXISTS order_items (  
id SERIAL PRIMARY KEY,  
order_id INT REFERENCES orders(id),  
product_id INT REFERENCES products(id),  
quantity INT NOT NULL,  
unit_price NUMERIC(10,2) NOT NULL,  
discount NUMERIC(5,2) DEFAULT 0  
);
```

```
-- Индексы для ускорения JOIN'ов (базовые)
CREATE INDEX IF NOT EXISTS idx_orders_client_id ON orders(client_id);
CREATE INDEX IF NOT EXISTS idx_orders_order_date ON orders(order_date);
CREATE INDEX IF NOT EXISTS idx_order_items_order_id ON order_items(order_id);
CREATE INDEX IF NOT EXISTS idx_order_items_product_id ON order_items(product_id);
CREATE INDEX IF NOT EXISTS idx_products_category ON products(category);
```

```
-- Проверка создания
SELECT 'Tables created successfully' AS status;
```

seed

```
-- =====
```

```
-- Генератор тестовых данных для e-commerce
-- =====
```

```
-- Очищаем таблицы (если нужно перезапустить)
TRUNCATE order_items, orders, products, clients RESTART IDENTITY CASCADE;
```

```
-- 1. Клиенты (10 000 записей)
INSERT INTO clients (name, email, phone)
SELECT
'Client ' || i,
'client' || i || '@example.com',
'+7' || lpad((9000000000 + i)::text, 10, '0')
FROM generate_series(1, 10000) AS i;
```

```
-- 2. Товары (1 000 записей)
INSERT INTO products (name, price, category)
SELECT
'Product ' || i,
(random() * 9000 + 100)::numeric(10,2), -- цена от 100 до 9100
CASE (i % 5)
WHEN 0 THEN 'Electronics'
WHEN 1 THEN 'Books'
WHEN 2 THEN 'Clothing'
WHEN 3 THEN 'Home'
ELSE 'Sports'
END
FROM generate_series(1, 1000) AS i;
```

```
-- 3. Заказы (500 000 записей за последние 2 года)
INSERT INTO orders (client_id, order_date, status, total_amount)
SELECT
(random() * 9999 + 1)::int, -- случайный client_id от 1 до 10000
CURRENT_DATE - (random() * 730)::int, -- дата за последние 2 года
CASE (i % 4)
```

```

WHEN 0 THEN 'new'
WHEN 1 THEN 'paid'
WHEN 2 THEN 'shipped'
ELSE 'done'
END,
0 -- временно, пересчитаем ниже
FROM generate_series(1, 500000) AS i;

```

```

-- 4. Позиции заказа (в среднем 3 позиции на заказ = ~1.5М записей)
INSERT INTO order_items (order_id, product_id, quantity, unit_price, discount)
SELECT
o.id,
(random() * 999 + 1)::int, -- случайный product_id от 1 до 1000
(random() * 5 + 1)::int, -- количество от 1 до 5
p.price,
CASE WHEN random() < 0.2 THEN (random() * 20)::numeric(5,2) ELSE 0 END -- 20% заказов со
скидкой
FROM orders o
CROSS JOIN generate_series(1, 3) AS g -- 3 позиции на заказ
JOIN products p ON p.id = (random() * 999 + 1)::int;

```

```

-- 5. Пересчитываем total_amount в orders
UPDATE orders o
SET total_amount = (
SELECT COALESCE(SUM(oi.quantity * oi.unit_price * (1 - oi.discount/100)), 0)
FROM order_items oi
WHERE oi.order_id = o.id
);

```

```

-- Проверяем результат
SELECT
'clients' AS table_name, count(*) FROM clients
UNION ALL
SELECT 'products', count(*) FROM products
UNION ALL
SELECT 'orders', count(*) FROM orders
UNION ALL
SELECT 'order_items', count(*) FROM order_items;

```

analyze

```

--
=====
=====

-- ЭТАП 0: Подготовительные настройки для детального EXPLAIN ANALYZE
-- =====

```

```
-- Включаем вывод статистики буферов (показывает, сколько данных читалось с диска/из RAM)
SET track_io_timing = on;
```

```
-- =====
-- ЭТАП 1: БАЗОВЫЙ АНАЛИТИЧЕСКИЙ ЗАПРОС (До оптимизации)
-- Задача: "Топ-100 клиентов по выручке в категории 'Electronics' за последние 6 месяцев"
-- =====
\echo '=====
\echo 'ЭТАП 1: БАЗОВЫЙ ЗАПРОС (4 таблицы, 3 JOIN, без доп. индексов)'
\echo '====='
```

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
  c.id AS client_id,
  c.name,
  c.email,
  COUNT(DISTINCT o.id) AS order_count,
  SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)) AS total_revenue
FROM clients c
JOIN orders o ON o.client_id = c.id
JOIN order_items oi ON oi.order_id = o.id
JOIN products p ON p.id = oi.product_id
WHERE p.category = 'Electronics'
AND o.order_date >= CURRENT_DATE - INTERVAL '6 months'
GROUP BY c.id, c.name, c.email
ORDER BY total_revenue DESC
LIMIT 100;
```

```
-- =====
-- ЭТАП 2: ОПТИМИЗАЦИЯ 1 — ДОБАВЛЕНИЕ ЦЕЛЕВЫХ ИНДЕКСОВ
-- =====
\echo '=====
\echo 'ЭТАП 2: СОЗДАНИЕ ОПТИМИЗИРОВАННЫХ ИНДЕКСОВ'
\echo '====='
```

```
-- Составной индекс для фильтрации по дате и быстрого JOIN по клиенту
CREATE INDEX IF NOT EXISTS idx_orders_date_client ON orders(order_date, client_id);
```

```
-- Составной индекс для позиций заказа (сначала order_id для JOIN, потом product_id для фильтрации)
CREATE INDEX IF NOT EXISTS idx_order_items_order_product ON order_items(order_id, product_id);
```

```
-- Индекс для категории товара
CREATE INDEX IF NOT EXISTS idx_products_category ON products(category);
```

```
\echo 'Индексы созданы. Повторяем запрос...'
```

```

EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
c.id AS client_id, c.name, c.email,
COUNT(DISTINCT o.id) AS order_count,
SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)) AS total_revenue
FROM clients c
JOIN orders o ON o.client_id = c.id
JOIN order_items oi ON oi.order_id = o.id
JOIN products p ON p.id = oi.product_id
WHERE p.category = 'Electronics'
AND o.order_date >= CURRENT_DATE - INTERVAL '6 months'
GROUP BY c.id, c.name, c.email
ORDER BY total_revenue DESC
LIMIT 100;

```

```

-- =====
-- ЭТАП 3: ОПТИМИЗАЦИЯ 2 — ПЕРЕПИСЫВАНИЕ SQL И СОКРАЩЕНИЕ JOIN
(Денормализация)
-- =====

```

```

\echo '=====
\echo 'ЭТАП 3: ДЕНОРМАЛИЗАЦИЯ (Убираем JOIN с таблицей products)'
\echo '=====

```

```

-- 1. Добавляем категорию прямо в таблицу позиций заказа
ALTER TABLE order_items ADD COLUMN IF NOT EXISTS category VARCHAR(100);

```

```

-- 2. Заполняем это поле историческими данными (один раз)
UPDATE order_items oi
SET category = p.category
FROM products p
WHERE oi.product_id = p.id AND oi.category IS NULL;

```

```

-- 3. Создаем индекс по новой колонке
CREATE INDEX IF NOT EXISTS idx_order_items_category ON order_items(category);

```

```

\echo 'Денормализация завершена. Повторяем запрос БЕЗ JOIN products...'

```

```

EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
c.id AS client_id, c.name, c.email,
COUNT(DISTINCT o.id) AS order_count,
SUM(oi.quantity * oi.unit_price * (1 - oi.discount / 100.0)) AS total_revenue
FROM clients c
JOIN orders o ON o.client_id = c.id
JOIN order_items oi ON oi.order_id = o.id
-- JOIN products p удален!
WHERE oi.category = 'Electronics'
AND o.order_date >= CURRENT_DATE - INTERVAL '6 months'
GROUP BY c.id, c.name, c.email
ORDER BY total_revenue DESC

```

```
LIMIT 100;
```

```
-- =====
-- ЭТАП 4: ОПТИМИЗАЦИЯ 3 — ПАРТИЦИОНИРОВАНИЕ (Концепт и DDL)
-- =====
\echo '=====
\echo 'ЭТАП 4: ПАРТИЦИОНИРОВАНИЕ (DDL для справки)'
\echo '=====
-- Примечание: Полное пересоздание таблиц с данными в скрипте рискованно.
-- Здесь показана правильная структура для партиционирования по месяцам.
-- В реальном проекте данные мигрируют в такие таблицы при вставке.
```

```
CREATE TABLE IF NOT EXISTS orders_partitioned (
  id INT,
  client_id INT,
  order_date TIMESTAMP NOT NULL,
  status VARCHAR(20),
  total_amount NUMERIC(12,2)
) PARTITION BY RANGE (order_date);
```

```
-- Пример создания партии за один месяц
CREATE TABLE IF NOT EXISTS orders_2023_10 PARTITION OF orders_partitioned
FOR VALUES FROM ('2023-10-01') TO ('2023-11-01');
```

```
\echo 'См. вывод выше. Партиционирование позволяет БД полностью игнорировать'
\echo 'партии, не попадающие в диапазон WHERE (Partition Pruning).'
\echo '=====
```

```
-- =====
-- ЭТАП 5: СВОДНАЯ СТАТИСТИКА ДЛЯ ОБОСНОВАНИЯ
-- =====
\echo '=====
\echo 'ЭТАП 5: СВОДНАЯ ИНФОРМАЦИЯ ПО РАЗМЕРАМ И ИНДЕКСАМ'
\echo '=====
```

```
SELECT
  relname AS table_name,
  pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
  pg_size_pretty(pg_relation_size(relid)) AS data_size,
  pg_size_pretty(pg_total_relation_size(relid) - pg_relation_size(relid)) AS index_size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

=====

ЭТАП 4: ПАРТИЦИОНИРОВАНИЕ (DDL для справки)

=====

CREATE TABLE

CREATE TABLE

См. вывод выше. Партиционирование позволяет БД полностью игнорировать  
партиции, не попадающие в диапазон WHERE (Partition Pruning).

=====

=====

ЭТАП 5: СВОДНАЯ ИНФОРМАЦИЯ ПО РАЗМЕРАМ И ИНДЕКСАМ

=====

| table_name     | total_size | data_size | index_size |
|----------------|------------|-----------|------------|
| order_items    | 390 MB     | 181 MB    | 209 MB     |
| orders         | 111 MB     | 56 MB     | 55 MB      |
| clients        | 1864 kB    | 912 kB    | 952 kB     |
| products       | 152 kB     | 64 kB     | 88 kB      |
| orders_2023_10 | 0 bytes    | 0 bytes   | 0 bytes    |

(5 rows)